Министерство науки и высшего образования
Российской Федерации

Университет ИТМО

Факультет программной инженерии и
компьютерной техники

Информационные системы

Лабораторная работа № 1

Разработка информационной системы для управления объектами

класса `HumanBeing`

Исполнитель: Ясаков А.

Преподаватель: Кобик Н. А.

Санкт-Петербург

2026

Содержание

1 Текст задания **2**

1.1 Описание классов предметной области 2

1.2 Требования к функционалу 2

1.3 Специальные операции 3

1.4 Требования к хранению и технологиям 3

2 UML-диаграммы **5**

2.1 Диаграмма классов 5

2.2 Диаграмма пакетов 6

3 Исходный код **8**

3.1 Стек технологий 8

3.2 Структура проекта 8

3.3 Схема базы данных 9

4 Выводы **10**

1 Текст задания

Реализовать информационную систему, которая позволяет взаимодействовать с объектами класса `HumanBeing`.

1.1 Описание классов предметной области

```
public class HumanBeing {
    private Integer id;          // not null, >0, unique, auto-generated
    private String name;         // not null, not empty
    private Coordinates coordinates; // not null
    private java.time.LocalDateTime creationDate; // not null, auto-
        generated
    private boolean realHero;
    private boolean hasToothpick;
    private Car car;             // nullable
    private Mood mood;           // nullable
    private float impactSpeed;   // max 738
    private WeaponType weaponType; // not null
}

public class Coordinates {
    private Float x;             // not null, max 256
    private float y;             // > -533
}

public class Car {
    private String name;         // nullable
}

public enum Mood {
    SADNESS, APATHY, CALM, FRENZY;
}

public enum WeaponType {
    PISTOL, KNIFE, BAT;
}
```

1.2 Требования к функционалу

1. CRUD-операции в отдельных интерфейсах: создание, получение по ID (с выводом связанных объектов), обновление, удаление.

2. При создании — возможность связать объект с существующими `Coordinates` и `Car`.

3. Все операции на серверной стороне, синхронизация с БД.

4. Главный экран: таблица со всеми атрибутами, пагинация.

5. Фильтрация и сортировка по строковым колонкам (только полное совпадение).

6. Переход к редактированию из таблицы и из области детального просмотра.

7. Real-time синхронизация между клиентами при добавлении/изменении/удалении.

8. При удалении — проверка связанных объектов, отмена с сообщением при наличии связей.

9. Информативные сообщения об ошибках валидации.

1.3 Специальные операции

1. Количество объектов с `impactSpeed` равным заданному.

2. Количество объектов с `weaponType` меньше заданного.

3. Количество объектов с `impactSpeed` больше заданного.

4. Удалить всех героев без зубочисток.

5. Пересадить всех героев без машины на красные «Lada Kalina».

Спецоперации реализуются в бизнес-логике без прямого использования функций и процедур БД.

1.4 Требования к хранению и технологиям

- СУБД: PostgreSQL, БД `studs`, хост `pg`.

- Все ограничения полей — на уровне ORM и БД. ID генерируется средствами БД.
- Меню для навигации между логическими частями системы.
- Стек: Spring Boot, JPA + Hibernate, разделение на слои.

2 UML-диаграммы

2.1 Диаграмма классов

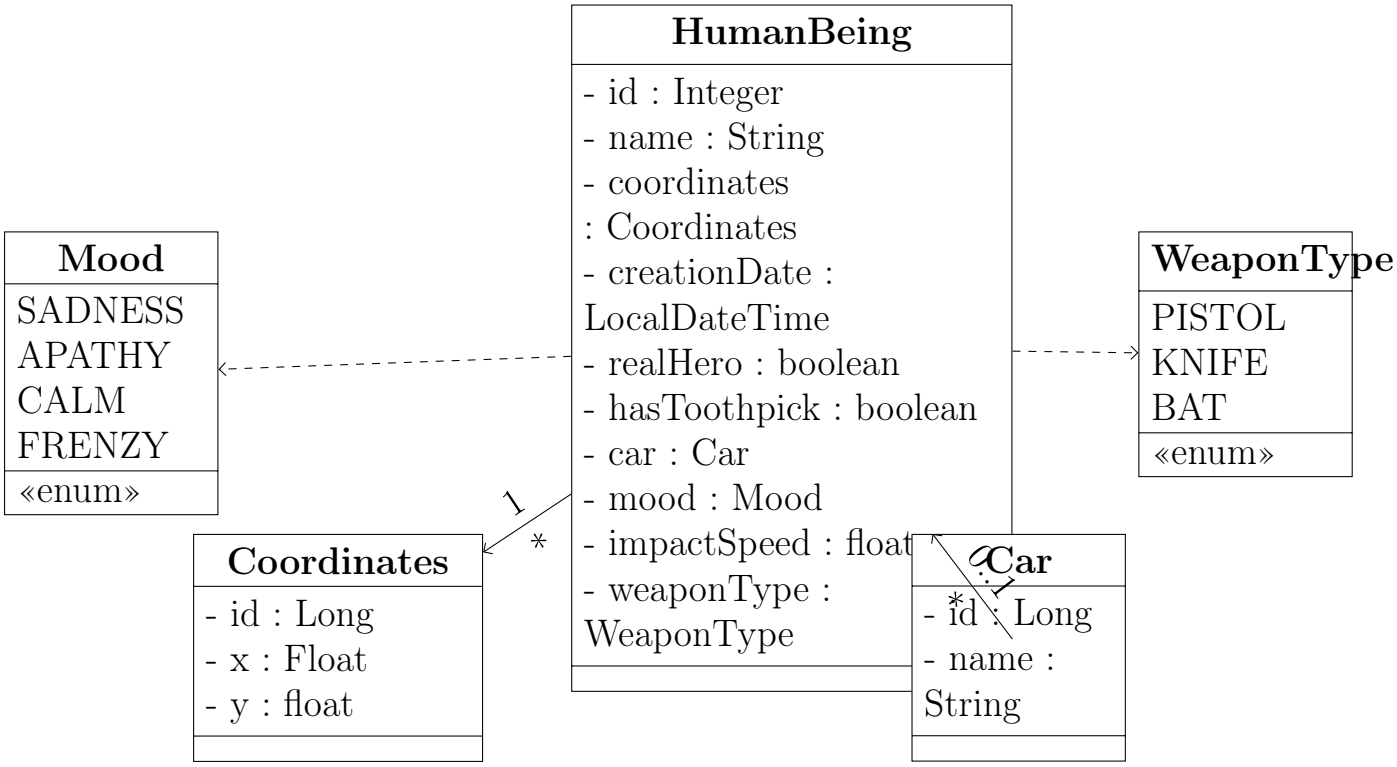


Рис. 1: Диаграмма классов предметной области

2.2 Диаграмма пакетов

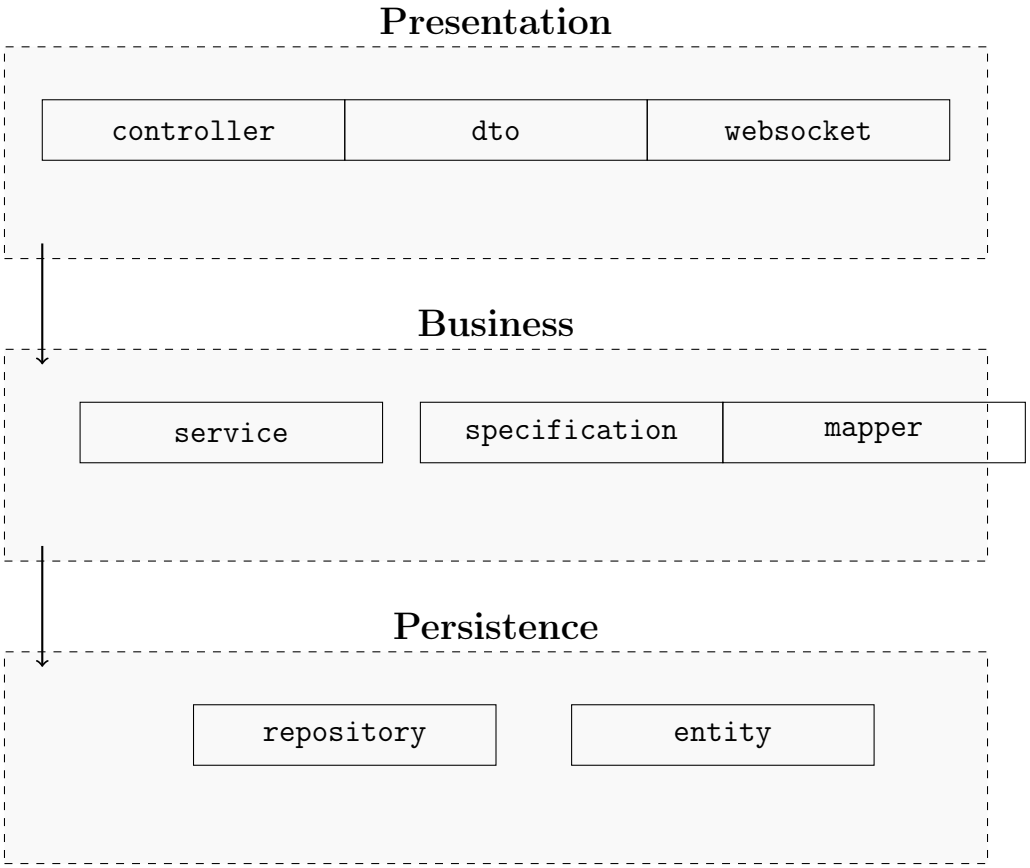


Рис. 2: Диаграмма пакетов приложения

Описание пакетов:

- `controller` — REST-контроллеры (`HumanBeingController`, `ReferenceController`, `SpecialOperationsController`).
- `dto/request` — входные DTO с Bean Validation (`HumanBeingRequest`, `CoordinatesRequest`, `CarRequest`).
- `service` — бизнес-логика (`HumanBeingService`, `CoordinatesService`, `CarService`).
- `repository` — Spring Data JPA репозитории.
- `entity` — JPA-сущности и перечисления (`HumanBeing`, `Coordinates`, `Car`, `Mood`, `WeaponType`).
- `mapper` — преобразование DTO в сущности.

- `specification` — динамические фильтры через `JpaSpecificationExecutor`.
- `websocket` — STOMP-брокер, события `EntityChangedEvent`, transactional listeners.
- `config` — конфигурация WebSocket.
- `exception` — глобальный обработчик исключений, кастомные исключения.

3 Исходный код

Исходный код проекта доступен в репозитории GitHub:
https://github.com/yaart/IS_lab1

3.1 Стек технологий

- **Backend:** Java 17, Spring Boot 3.4.4, Spring Data JPA, Hibernate, Spring WebSocket
- **Frontend:** React 18, Vite, Material UI (MUI), STOMP over WebSocket
- **База данных:** PostgreSQL 18 (docker-compose), хост `pg`, БД `studs`
- **Сборка:** Maven (backend), Vite (frontend)

3.2 Структура проекта

```
is-lab1-spring/
  pom.xml
  docker-compose.yml
  src/main/java/com/example/islab1/
    IsLab1Application.java
    config/WebSocketConfig.java
    controller/
      SpaController.java
      api/
        HumanBeingController.java
        ReferenceController.java
        SpecialOperationsController.java
    dto/request/
      CarRequest.java
      CoordinatesRequest.java
      HumanBeingRequest.java
    entity/
      Car.java, Coordinates.java, HumanBeing.java
      Mood.java, WeaponType.java
    exception/
      ApiErrorResponse.java, BusinessException.java
      GlobalExceptionHandler.java, ResourceNotFoundException.java
      ValidationErrorResponse.java
    mapper/HumanBeingMapper.java
    repository/
      CarRepository.java, CoordinatesRepository.java
```

```
      HumanBeingRepository.java
    service/
      CarService.java, CoordinatesService.java
      HumanBeingService.java
    specification/HumanBeingSpecification.java
    websocket/
      ChangeType.java, EntityChangedEvent.java
      EntityChangedListener.java, EntityChangePublisher.java
  src/main/resources/
    application.yml, application-dev.yml
  frontend/
    src/App.jsx
    src/components/
      HumansTable.jsx, HumanDialog.jsx
      SpecialOps.jsx, ConfirmDialog.jsx
    src/api.js, src/websocket.js, src/theme.js
```

3.3 Схема базы данных

```sql
CREATE TABLE coordinates (
    id BIGSERIAL PRIMARY KEY,
    x FLOAT NOT NULL CHECK (x <= 256),
    y FLOAT NOT NULL CHECK (y > -533)
);

CREATE TABLE car (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(255)
);

CREATE TABLE human_being (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL CHECK (char_length(trim(name)) > 0),
    coordinates_id BIGINT NOT NULL REFERENCES coordinates(id),
    creation_date TIMESTAMP NOT NULL,
    real_hero BOOLEAN NOT NULL DEFAULT FALSE,
    has_toothpick BOOLEAN NOT NULL DEFAULT FALSE,
    car_id BIGINT REFERENCES car(id),
    mood VARCHAR(20),
    impact_speed FLOAT CHECK (impact_speed <= 738),
    weapon_type VARCHAR(20) NOT NULL
);
```

4 Выводы

В ходе выполнения лабораторной работы была разработана информационная система для управления объектами класса `HumanBeing`. Система реализует полный набор CRUD-операций, фильтрацию и сортировку таблицы, real-time синхронизацию между клиентами через WebSocket, пять специальных операций и защиту от некорректного удаления связанных объектов.

Применён трёхслойный архитектурный подход (контроллеры, сервисы, репозитории) с чётким разделением ответственности. Все ограничения полей классов дублированы на уровнях Java-валидации, ORM и базы данных, что обеспечивает целостность данных на всех уровнях приложения.

Для клиентской части использован стек React + Material UI с серверной пагинацией и мгновенным обновлением данных через STOMP-протокол. Навигация между таблицей объектов и специальными операциями реализована через меню с вкладками.